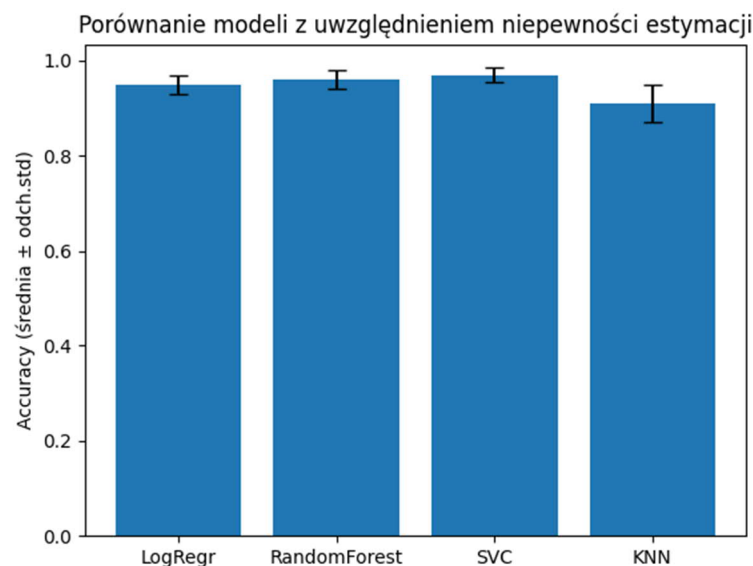


Niepewność estymatorów i rola cross-validation w jej ocenie

W statystyce i uczeniu maszynowym często musimy oszacować nieznane parametry na podstawie danych obserwacyjnych, np. średnią, wariancję czy parametry modelu regresji. Taki obliczony parametr nazywamy **estymatorem**. Ponieważ estymator opiera się na danych, które są próbą losową, jego wartość **nie jest stała** — zmieniałaby się, gdybyśmy zebrali inne dane. Tę zmienność nazywamy **niepewnością estymatora**.

Wykres słupkowy (bar plot) z belkami błędów (error bars)



```
models = ["LogRegr", "RandomForest", "SVC", "KNN"]
```

```
means = [0.95, 0.96, 0.97, 0.91]
```

```
stds = [0.02, 0.02, 0.015, 0.04]
```

Każdy model ma słupek pokazujący średnią **accuracy** /dokładność/, a pionowe kreski symbolizują odchylenie standardowe.

Co wpływa na niepewność estymatorów?

- wielkość próby — mała próbka to duża zmienność
- szum i błędy pomiaru

- jakość i reprezentatywność danych
- złożoność modelu i przeuczenie (overfitting)

Dlatego potrzebujemy metod oceny stabilności estymatora.

Cross-validation jako sposób oceny niepewności

Cross-validation (walidacja krzyżowa) to metoda polegająca na wielokrotnym dzieleniu danych na część treningową i testową oraz mierzeniu jakości modelu na różnych fragmentach danych.

Jak działa k-fold cross-validation?

1. Dzielimy dane na *k* równych części (tzw. folds).
2. Trenujemy model *k* razy — za każdym razem używając innej części jako testowej.
3. Liczymy błędy dla każdej iteracji i obliczamy średnią oraz odchylenie standardowe.

$$CV_error = \frac{1}{k} \sum_{i=1}^k error_i$$

gdzie:

- $error_i$ to błąd (lub odwrotnie dokładność) dla *i*-tej iteracji,
- *k* to liczba foldów.

Co dokładnie robimy w kroku nr 3?

1. Trenujemy model na danej części zbioru danych (fold).
2. Testujemy go na pozostałej części.
3. Obliczamy wynik jakości (np. accuracy) dla danego folda.
4. Powtarzamy ten proces dla wszystkich foldów (np. 5 lub 10 razy).
5. Otrzymujemy listę wyników z każdej iteracji, np.:

```
accuracy = [0.96, 0.95, 0.97, 0.94, 0.96]
```

Po co liczymy średnią wartość wyników?

Średnia mówi nam, **jaki jest oczekiwany wynik modelu na nowych danych**. Nie opiera się na jednym losowym podziale, ale na wielu testach.

Przykład:

$$\text{mean_accuracy} = \frac{0.96 + 0.95 + 0.97 + 0.94 + 0.96}{5} = 0.956$$

Czyli model średnio osiąga **95.6% skuteczności**.

Po co liczymy odchylenie standardowe?

Odchylenie standardowe mierzy **rozrzut wyników między poszczególnymi iteracjami**.

$$\text{std} = \sqrt{\frac{1}{k} \sum (\text{score}_i - \text{mean})^2}$$

Jeśli **std jest małe**, wyniki są podobne w każdej iteracji to model jest **stabilny i przewidywalny**.

Jeśli **std jest duże**, model daje bardzo różne wyniki to **jest niepewny i niestabilny**.

Co daje cross-validation?

CEL	WYJAŚNIENIE
MIERZY NIEPEWNOŚĆ ESTYMACJI	pokazuje, jak model działa na różnych podzbiorach danych
ZAPOBIEGA PRZEUCZENIU (OVERFITTING)	model nie uczy się tylko na jednym zbiorze
POZWALA PORÓWNAĆ MODELE	niższy błąd CV to stabilniejszy model
POZWALA OPTYZMALIZOWAĆ HIPERPARAMETRY	tuning modeli wykonywany w środku CV

Uwaga: W uczeniu maszynowym **hiperparametry** to ustawienia modelu, które **nie są uczenia automatycznie**, ale muszą być **wybrane przez użytkownika**.

Przykłady hiperparametrów:

- liczba sąsiadów w KNN (n_neighbors)
- liczba drzew w Random Forest (n_estimators)

- współczynnik C w SVM (C)
- głębokość drzewa (max_depth)
- tempo uczenia w sieciach neuronowych (learning_rate)

Te wartości mogą bardzo wpływać na jakość modelu.

Dlatego musimy znaleźć **najlepszą kombinację hiperparametrów**, która daje najwyższą skuteczność.

Dlaczego cross-validation pomaga w ocenie niepewności estymatorów?

Bez CV wynik modelu zależy od jednego podziału danych (train/test).

Z CV widzimy **rozrzut wyników**, czyli miarę niepewności:

- małe odchylenie błędów to model jest stabilny i estymatory mało zależą od próby
- duże odchylenie to model niestabilny, wrażliwy na dane

Przykład interpretacji

MODEL	ŚREDNI BŁĄD CV	ODCHYLENIE	WNIOSKI
MODEL A	0.12	0.01	stabilny
MODEL B	0.10	0.09	niepewny, nadmiernie dopasowany

W praktyce zwykle wybiera się **model o mniejszym błędzie, ale stabilny**, nie zaś ten, który ma najniższy błąd pojedynczej estymacji.

Poniżej znajduje się przykładowy kod w Pythonie, który pokazuje **cross-validation**, oblicza **średni błąd modelu oraz odchylenie standardowe**, co ilustruje **niepewność estymacji**.

```
import numpy as np
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.datasets import make_regression

# Tworzymy przykładowy zbiór danych (100 próbek, 1 zmienna)
X, y = make_regression(n_samples=100, n_features=1, noise=15, random_state=42)

# Model regresji liniowej
model = LinearRegression()

# K-fold cross validation (k=5)
k = 5
kf = KFold(n_splits=k, shuffle=True, random_state=42)
```

```
# Liczymy MSE z cross validation (ujemne wartości oznaczają błąd, dlatego
mnożymy przez -1)
scores = cross_val_score(model, X, y, cv=kf, scoring='neg_mean_squared_error')
mse_scores = -scores # zmiana na dodatnie MSE

print("Wyniki MSE z kolejnych foldów:", mse_scores)
print("Średni błąd MSE:", np.mean(mse_scores))
print("Odchylenie standardowe błędu:", np.std(mse_scores))
```

Co robi kod?

Element	Cel
make_regression()	generuje dane do testów
LinearRegression()	prosty model
KFold()	dzieli dane na 5 części
cross_val_score()	trenuje model 5 razy i oblicza błąd
mean()	średni błąd modelu
std()	niepewność estymatora